

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «КубГУ»)

Факультет компьютерных технологий и прикладной математики
Кафедра информационных технологий

Допустить к защите
Заведующий кафедрой
док. физ.-мат. наук, доц.
_____ В.А. Бабешко
(подпись)
_____ 2026 г.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
(МАГИСТЕРСКАЯ РАБОТА)

РАЗРАБОТКА ПРИЛОЖЕНИЯ ДЛЯ ОЦЕНКИ ОБЛИГАЦИЙ

Работу выполнил _____ В.Р. Мошняков
(подпись)

Направление подготовки 01.04.02 Прикладная математика и информатика

Направленность Математическое моделирование в естествознании и
технологиях

Научный руководитель
канд. ф.-м. наук, _____ А.А. Евдокимов
(подпись)

Нормоконтролер
канд. ф.-м. наук, доц. _____ С.Е. Рубцов
(подпись)

Краснодар
2026

РЕФЕРАТ

Выпускная квалификационная работа 58 с., 16 рис., 24 источника.

ДОЛГОВЫЕ ЦЕННЫЕ БУМАГИ, ОБЛИГАЦИЯ, ИНФОРМАЦИОННО-АНАЛИТИЧЕСКАЯ СИСТЕМА, ИАС, ПОРТФЕЛЬ ОБЛИГАЦИЙ, РИСК ПОРТФЕЛЯ, ВЕБ-ПРИЛОЖЕНИЕ, PYTHON

Цель работы: спроектировать и реализовать веб-приложение для моделирования цен и расчета доходности облигаций, которое предоставит розничным инвесторам расширенные аналитические возможности, включая функцию интерактивного макроэкономического стресс-тестирования.

В процессе работы были изучены: математические модели ценообразования долговых ценных бумаг, алгоритмы сценарного стресс-тестирования портфелей на основе показателей дюрации и выпуклости, а также современные архитектурные паттерны проектирования финансовых веб-приложений и методы интеграции с биржевыми программными интерфейсами.

В практической части была спроектировано и разработано полноценное веб-приложение для риск-аналитики облигационных портфелей. В ходе реализации была настроена реляционная база данных, написан конвейер автоматического сбора котировок с Московской биржи и разработано вычислительное ядро. Ключевым результатом стал запуск модуля интерактивного стресс-тестирования, который через современный пользовательский интерфейс визуализирует влияние изменения процентных ставок на капитал инвестора.

Средства разработки: язык программирования Python, язык программирования TypeScript, фреймворки FastAPI, React, pytest, Tailwind CSS, библиотеки NumPy, Pandas, SciPy, Shadcn UI, Pydantic, SQLAlchemy, asynpg, httpx, asyncio, APScheduler, logging, axios, база данных PostgreSQL.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	5
1 Анализ предметной области.....	7
1.1 Рынок долговых ценных бумаг РФ.....	7
1.2 Сравнительный анализ существующих финтех решений.....	9
1.3 Математические модели ценообразования облигаций.....	12
1.4 Оценка процентного риска и стресс-тестирования.....	14
2 Архитектура системы.....	18
2.1 Требования к системе и выбор технологического стека.....	18
2.2 Проектирование базы данных.....	22
2.3 Архитектура приложения и интеграция с внешними сервисами и системами.....	23
3 Программная реализация.....	28
3.1 Разработка логики.....	28
3.1.1 Доступ к данным БД.....	29
3.1.2 Математический блок.....	32
3.1.3 Интеграция с Московской Биржей и обработка данных.....	35
3.1.4 Портфели и стресс-тестирование портфеля.....	37
3.1.6 Программный интерфейс API.....	39
3.1.7 Автоматизация синхронизации данных.....	41
3.2 Реализация пользовательского интерфейса.....	43
3.2.1 Скринер и страница облигации.....	44
3.2.2 Портфель.....	47
3.3 Тестирование системы.....	51

ЗАКЛЮЧЕНИЕ.....	54
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	56

ВВЕДЕНИЕ

В последние несколько лет российский фондовый рынок переживает масштабный приток розничных инвесторов. На фоне высокой волатильности рынка акций многие пользователи обращаются к долговым ценным бумагам как к более консервативному и предсказуемому инструменту. Однако текущая макроэкономическая ситуация и частые изменения ключевой ставки Банка России обнажили серьезную проблему: подавляющее большинство частных инвесторов не умеют или не имеют возможности оценивать процентный риск своих портфелей. Популярны мобильные приложения крупнейших брокеров предоставляют лишь базовую информацию, такую как текущая цена и номинальная доходность, намеренно упрощая интерфейс и скрывая сложную аналитику. Профессиональные же торговые терминалы отличаются высоким порогом входа и недоступны массовому пользователю. В результате возникает острая нехватка легковесных и понятных программных инструментов, способных переводить сложный риск-менеджмент на язык рядового инвестора.

В связи с этим объектом данного исследования выступает процесс оценки стоимости и процентного риска портфелей долговых ценных бумаг, а предметом исследования являются математические модели, алгоритмы и архитектурные решения, необходимые для автоматизации этой оценки. Главная цель работы заключается в проектировании и реализации программного веб-приложения для моделирования цен и расчета доходности облигаций, которое предоставит розничным инвесторам расширенные аналитические возможности, включая функцию интерактивного макроэкономического стресс-тестирования.

Для достижения поставленной цели в ходе работы решается комплекс взаимосвязанных задач. В рамках работы были поставлены и решены следующие задачи:

- системный анализ предметной области, обзор существующих решений для розничных инвесторов и формализация математического аппарата дисконтирования денежных потоков облигаций;

- проектирование реляционной базы данных и разработка общей архитектуры приложения, разделяющей серверное вычислительное ядро, источник рыночных данных и клиентский интерфейс;

- программная реализация серверной части с подсистемами сбора рыночных котировок через API Московской биржи, расчёта доходности к погашению численным методом Ньютона, стресс-тестирования портфеля методом разложения Тейлора;

- разработка пользовательского интерфейса на связке фреймворка React и языка TypeScript, обеспечивающего визуализацию позиций, календаря купонных выплат, результатов стресс-теста;

- автоматизированное модульное и нагрузочное тестирование вычислительного ядра, подтверждающее корректность аналитических расчётов;

Методологической основой работы является комплексный подход, включающий анализ научно-технической литературы по теме, проектирование методами системного и структурного анализа и программную реализацию алгоритмов, а также применением автоматизированного модульного, интеграционного и нагрузочного тестирования. Работа состоит из трёх основных разделов, охватывающих анализ и теоретическую обзор, проектирование системы, процесс разработки и тестирование.

1 Анализ предметной области

1.1 Рынок долговых ценных бумаг РФ

Облигация – это долговой инструмент, обязывающий эмитента (называемого также должников или заемщиком) в течении установленного промежутка времени выплатить кредитору/инвестору взятую в займы сумму плюс процент [1]. Для классической облигации на фондовом рынке РФ обязательно устанавливаются:

- номинал/долг
- фиксированная дата возврата долга (номинал)
- порядок и условия выплаты дохода

Номиналом или номинальной стоимостью называется сумма долга в валюте, которую эмитент берет у инвестора за одну облигацию. Дата возврата номинала носит название дата погашения. Однако для бессрочных облигаций дата погашения может не указываться. Доходом облигации является ежегодным процентом от номинальной стоимости, выплачиваемый за использование деньгами заемщика. Такие выплаты называются купонными или просто купонами, а процент – купонной ставкой. Периодические купонные выплаты являются обязательными для всех типов облигаций, кроме одного. Тип облигаций, у которых нет купонов называется соответственно бескупонным. У таких облигаций доходный процент или нулевой купон получается за счет покупки облигации по более низкой цене от номинала и выплачивается при погашении [1][2][3].

На долговом рынке цены облигаций на торгах указываются не в денежных эквивалентах, а в процентах от её номинала. Номинал не имеет постоянного общего значения для всех облигаций и устанавливается эмитентом по прописанным в законах РФ правилам. Поэтому такой подход позволяет инвесторам сравнивать бумаги с разными номиналами. Но этот процент, указывающийся на биржах для инвесторов обычно является

неполной ценой бумаги и называется чистая цена. Чаще всего при покупке облигации у участника торгов инвестор платит другую сумму. Механизм, который помогает рассчитать полную цену (грязную цену) называется накопленный купонный доход. Он служит компенсацией дохода по купону пропорционально количеству дней, которые заемщик владел облигацией в течении текущего купонного периода – эта сумма называется НКД.

Важно рассмотреть основные свойства типов облигаций и их основных представителей на фондовом рынке. Эмитенты классифицируются на:

- ОФЗ (облигации федерального займа) – государственные облигации, выпускаются Минфин РФ, считаются наиболее стабильные и надежные;
- муниципальные облигации – долговые бумаги регионов и муниципалитетов;
- корпоративные облигации – выпускаются коммерческими организациями;

Но более важным свойством облигаций, которое влияет на большинство аналитических расчетов, является тип купонных выплат. *Облигации с фиксированным купоном* работают по принципу, в котором ставка и размеры купона указываются и известны вплоть до даты погашения. Противоположным типом являются *облигации с плавающим купоном (флоатер)*. Их особенностью является отсутствие фиксированного размера купона, он высчитывается по указанной эмитентом формуле при эмиссии бумаги. Такой тип считается более устойчивым к процентным рискам, что доказывает увеличение их доли на рынке в период жесткой денежно-кредитной политики Банка России. Другой надежной облигацией является *линкеры (инфляционные облигации)*. Они имеют «фиксированные» купоны, но номинал таких облигаций ежедневно пересчитывается на величину инфляции. *Облигации с амортизацией долга* — это бумаги номинал которых возвращается инвестору не одним платежом в конце срока погашения, а выплачивается частями вместе с купонами. Амортизация кардинально влияет

на расчет купонов в будущих периодах, так как процент отсчитывается на уменьшенный остаток номинала [4].

Текущим лидером и монополистом в сфере торгов на фондовом рынке является ПАО «Московская Биржа». Что позволяет однозначно определить оптимального поставщика данных для разрабатываемой системы. Критически важным является получение актуальных данных из первоисточника, что и предоставляет Мосбиржа. Большинство других финансовых систем предоставляют данные, полученные с серверов Московской биржи. Извлечение данных из других систем может привести к серьезным ошибкам. Ещё одним достоинством биржи является открытость и доступность данных для всех. Более подробно о том каким способом извлекаются данные будет описано в главах 2 и 3 [5].

1.2 Сравнительный анализ существующих финтех решений

Для выявления функциональных и архитектурных требований к разрабатываемой системе был проведен анализ актуальных и популярных финансовых решений на российском рынке. Все существующие программные средства можно разделить на три основные группы по их целевым пользователям и глубине рыночной аналитики: брокерские приложения (B2C – Business-to-Consumer) – Т-Инвестиции, Сбер Инвестиции, Альфа Инвестиции и другие, информационно-аналитические порталы – Smart-lab, Cbonds и профессиональные торговые терминалы (B2B – Business-to-Business). Рассмотрим их функциональные, нефункциональные и целевые различия.

Основной задачей группы брокерских приложений, в основном мобильных, является максимальное упрощение процесса торговли. Их основными особенностями являются предельно понятный пользовательский интерфейс и простота покупки и продажи ценных бумаг. Такой подход позволяет скрыть сложные показатели и метрики рисков, оставив

пользователю базовую информацию (номинал, дата погашения, текущая чистая цена и простая доходность к погашению). Часто в приложениях отсутствуют функционалы расчета модифицированной дюрации и выпуклости, не отображают оценок риска портфеля, нет возможности проводить сценарные моделирования портфеля/ей.

Во вторую группу входят ресурсы, в основном веб-сайты, предназначенные для анализа финансовых показателей и инструментов. Они предоставляют обширные показатели и детализированные данные по ценным бумагам. На таких ресурсах зачастую отсутствует торговля и ведение портфеля пользователя, что не позволяет анализировать показатели и данные совокупности бумаг. Рассматривая порталы Smart-lab и Cbonds можно выделить старомодный дизайн и подписочную систему для работы соответственно.

Последняя группа представлена сложными системами для юридических лиц, а не массового пользователя. Совмещает в себе торговлю и инструменты анализа. Главным недостатком для обычного пользователя таких продуктов является высокий порог входа, ценообразование, избыточный функционал.

Как видно из сравнительной таблицы 1.2 текущие решения не предоставляют инвестору-обывателю функционала стресс-тестирования портфеля и прогноза по историческим показателям портфеля. Брокерские приложения сфокусированы на торговле, поэтому им невыгодно перегружать клиента информацией о рисках снижения стоимости его портфеля при изменении макроэкономических параметров. В то же время, профессиональные терминалы требуют навыков работы с ними и материальных затрат [6][7].

Таблица 1.2 – Функционал финансовых систем по группам

Функциональный критерий	Брокерские приложения	Информативно-аналитический портал	Профессиональный терминал
Скринер и фильтрация	Есть	Есть	Есть
Расчет УТМ и НКД	Есть	Есть	Есть
История торговли (PnL)	Есть	Нет	Есть
Расчет дюрации и выпуклости	Нет	Есть	Есть
Удобство пользовательского интерфейса	Высокое	Средне	Низкое
Портфельные риски	Нет	Нет	Есть
Стресс-тест портфеля	Нет	Нет	Есть
Доступность/ Стоимость	Бесплатно	Бесплатно/ Подписка	Платная лицензия

Таким образом, можно сформировать четкие функциональные требования к разрабатываемой системе, которая сочетает в себе положительные стороны каждой из групп: доступность рядовому инвестору, современный и удобный пользовательский веб-интерфейс, детализированные показатели по портфелю и по отдельным облигациям, добавление расчета рисков портфеля.

1.3 Математические модели ценообразования облигаций

Важной частью логики системы является математический блок, рассчитывающий цену облигаций, риски, выпуклость и другие метрики портфеля. Базовой метрикой для любой облигации на фондовых рынках является цена облигации. Цены облигаций существуют двух видов: чистая и грязная, и соответственно рассчитываются разными методами.

Чистая цена облигации котируется в процентах от номинала и не учитывает накопленный купонный доход. Поэтому расчеты доходности и рисков производятся с учетом грязной цены облигации.

$$P_{dirty} = \frac{P_{clean}}{100} \times N + ACI \quad (1.3.1)$$

где

N – номинальная стоимость облигации

ACI – накопленный купонный доход на дату расчетов

Зная цену облигации, можно вычислить один из её главных показателей – доходность к погашению YTM . Эта доходность представляет собой процентную ставку, при которой приведенная стоимость денежных потоков текущей инвестиции равна грязной цене облигации [1].

$$P = \sum_{i=1}^n \frac{CF_i}{(1+y)^{t_i}} \quad (1.3.2)$$

где

P – текущая цена облигации

CF_i – денежный поток за момент времени t

y – доходность к погашению

t_i – время до выплаты i -го денежного потока

n – число периодов

Для облигаций с количеством выплат больше двух такое уравнение не имеет точного аналитического решения. Поэтому его приближенное решение ищется с помощью численных алгоритмов. Простейшая аппроксимация доходности решается делением среднего годового дохода на среднюю цену – формула (1.3.3). Однако этот метод имеет крайне низкую точность, игнорируются параметры процента реинвестирования и точные даты из-за чего результат решения может иметь большую погрешность.

$$YTM \approx \frac{C + \frac{N - P}{t}}{\frac{N + P}{2}} \quad (1.3.3)$$

где

C – годовая купонная выплата (сумма всех купонных платежей за год)

N – номинальная стоимость купона

P – текущая рыночная цена облигации

t – срок до погашения, год

Метод половинного деления или дихотомия является надежным алгоритмом поиска доходности, но его скорость вычисления — это единственный недостаток.

Для большинства классических облигаций подходящим и оптимизированным по затрачиваемым ресурсам является метод Ньютона-Рафсона. Особенность данного метода для долгового рынка заключается в том, что первая производная функции от доходности тесно связана с показателем модифицированной дюрации. Однако этот метод не подходит для специфичных облигаций с нестандартными графиками выплат или при экстремальных ценовых шоках.

В современных программных математических ядрах часто применяется метод Брента, который сочетает в себе надежность дихотомии и скорость, сопоставимую с методом Ньютона-Рафсона. Для нахождения доходности к

погашению метод не требует вычисления производной. На каждой итерации метода динамически выбирается один из трех алгоритмов, исходя из поведения функции: обратная квадратичная интерполяция, метод секущих, метод половинного деления (на случай сбоев в первых двух) [8].

1.4 Оценка процентного риска и стресс-тестирования

Помимо расчета доходности, ключевой задачей системы является представление рисков целевому пользователю. Одним из основных показателей является процентный риск – вероятность изменения стоимости облигации в результате колебаний рыночных процентных ставок, которые тесно связаны с изменениями ключевой ставки ЦБ РФ. Между рыночной стоимостью облигации с фиксированным купоном и процентными ставками существует обратная зависимость: при росте ставок цена существующих выпусков падает, чтобы их доходность соответствовала новым, более высоким рыночным реалиям, и наоборот. Однако разные облигации реагируют на одно и то же изменение ставки по-разному.

Рассмотрим количественные показатели риска в финансовой математике. Для его измерения используется концепция дюрации, предложенная Фредериком Маколеем. Дюрация Маколея представляет собой средневзвешенный срок до получения инвестором всех денежных потоков по облигации, где в качестве весов выступают приведенные стоимости этих потоков [9].

$$D_{mac} = \frac{1}{P} \sum_{i=1}^n t_i \cdot \frac{CF_i}{(1+y)^{t_i}} \quad (1.4.1)$$

где,

P – грязная цена облигации

y – доходность к погашению

CF_i – i -тый денежный поток

t_i – время до выплаты потока, год

Сама по себе дюрация Маколея измеряется в временных показателях. Для прямого измерения чувствительности цены к изменению ставок вычисляется модифицированный показатель дюрации. Представляет собой значение в процентах и показывает на сколько изменится цена облигации при изменении её доходности на 1% или 100 базисных пунктов.

$$D_{mod} = \frac{D_{mac}}{1 + y} \quad (1.4.2)$$

где,

D_{mac} – дюрация Маколея

y – доходность к погашению

Именно эта метрика используется для простой линейной аппроксимации риска, формула (1.4.3). Математически модифицированная дюрация является первой производной функции цены облигации по процентной ставке.

$$\frac{\Delta P}{P} \approx -D_{mod} \cdot \Delta y \quad (1.4.3)$$

где,

D_{mod} – модифицированная дюрация

Δy - величина процентного шока, выраженная в десятичных долях

Однако линейная аппроксимация работает точно лишь на незначительных изменениях ставок до 100 базисных пунктов. Это связано с тем, что дюрация — это линейная аппроксимация (касательная к кривой цена-доходность в текущей точке). На самом же деле зависимость цены от доходности является нелинейной, выпуклой по отношению к началу координат. При сильных макроэкономических шоках, например изменение ключевой ставки на 5% пунктов, использование одной дюрации в расчетах

приводит к значительной погрешности оценки рисков. Для исправления этого недостатка используется выпуклость, которая математически является второй производной цены по доходности. Выпуклость показывает изменения степень изменения цены облигации при изменении ставки – чем больше выпуклость, тем меньше рост цены и обратно [10].

$$C = \frac{1}{P} \sum_{i=1}^n t_i(t_i+1) \frac{CF_i}{(1+y)^{t_i+2}} \quad (1.4.4)$$

где,

P – грязная цена облигации

y – доходность к погашению

CF_i – i -тый денежный поток

t_i – время до выплаты потока, год

В разрабатываемой информационно-аналитической системе стоит задача создания инструмента для интерактивного стресс-тестирования портфеля. Чтобы в режиме реального времени переоценивать сотни активов в зависимости от заданного пользователем шока ставок, использование полных перерасчетов методом дисконтирования неэкономично с точки зрения вычислительной нагрузки. Оптимальным инженерным и математическим решением выступает формула оценки изменения цены (1.4.5). В которой комбинируются первая производная (модифицированную дюрацию) и вторая производная (выпуклость), и которая позволяет с высокой точностью прогнозировать изменение цены при любых рыночных шоках:

$$\frac{\Delta P}{P} \approx -D_{mod} \cdot \Delta y + \frac{1}{2} C \cdot (\Delta y)^2 \quad (1.4.5)$$

где,

Δy - величина процентного шока, выраженная в десятичных долях

D_{mod} – модифицированная дюрация

С – выпуклость

Программный алгоритм системы должен агрегировать этот расчет на уровне всего портфеля. Для каждой позиции в портфеле рассчитывается её относительное изменение цены, после чего прогнозируемая стоимость стресс-позиции определяется формулой (1.4.6):

$$P_{stress} = P_{current} \times \left(1 + \frac{\Delta P}{P} \right) \quad (1.4.6)$$

где,

$P_{current}$ – текущая цена

Суммирование данных показателей по всем активам формирует итоговый прогноз состояния портфеля инвестора в рамках заданного макроэкономического сценария, что позволяет заблаговременно принимать решения о ребалансировке, например, сокращении дюрации или покупке флоатеров, до фактического изменения рыночных условий [1][8][10].

2 Архитектура системы

На основе анализа предметной области и приведенных математических расчётов, проведенных в первой главе, можно сформулировать четкие требования к разрабатываемой системе. Приоритетным компонентом финансовой системы является математический блок с точными и быстрыми алгоритмами вычисления, при ошибках в его вычислениях может привести к неверным инвестиционным решениям пользователя.

В данной главе с помощью различных методологий системного и структурного анализа будут формализованы функциональные и нефункциональные требования. На основе требований будет обоснован выбор технологического стека. Далее будет спроектирована схема базы данных и концептуальная логика взаимодействия компонентов системы.

2.1 Требования к системе и выбор технологического стека

Опираясь на ранее выявленные основные проблемы для различных групп финансовых систем, исходя из этого можно составить четкие требования к разрабатываемой системе. Взаимодействие с системой представлено в Use-Case диаграмме на рисунке 2.1.1, в которой изображены три основных актора: «Инвестор», «Системный планировщик» и внешний сервис «API Московской биржи». Инвестор – это классический пользователь системы. Системный планировщик выступает фоновым процессом или сервисом, который актуализирует данные без участия пользователя.

Инвестор должен иметь возможность поиска долговой бумаги по названию или идентификатору, с использованием фильтров и сортировок по необходимым пользователю параметрам. Такой инструмент в финансовой сфере называется скринером. После нахождения нужной облигации пользователю предоставляется подробная статичная информация о дате погашения, типе купона, денежным потокам и другие показатели. Если по

выданным показателям облигация удовлетворяет целям пользователя, он может добавить её в личный виртуальный портфель для дальнейшего анализа в динамике вместе с другими активами. Также у него должна быть возможность управлять портфелями и позициями в портфеле. Сводная статистика портфеля позволит пользователю получить больше информации по совокупным параметрам всех облигаций, а также оценивать влияние процентных рисков портфеля [11].

Роль системного планировщика заключается в периодической синхронизации данных ценных бумаг в системе с биржей – актором «API Московской биржи». Локальное хранение этих данных необходимо, чтобы устранить единую точку отказа работоспособности приложения и ускорил взаимодействие компонентов системы [12][13].

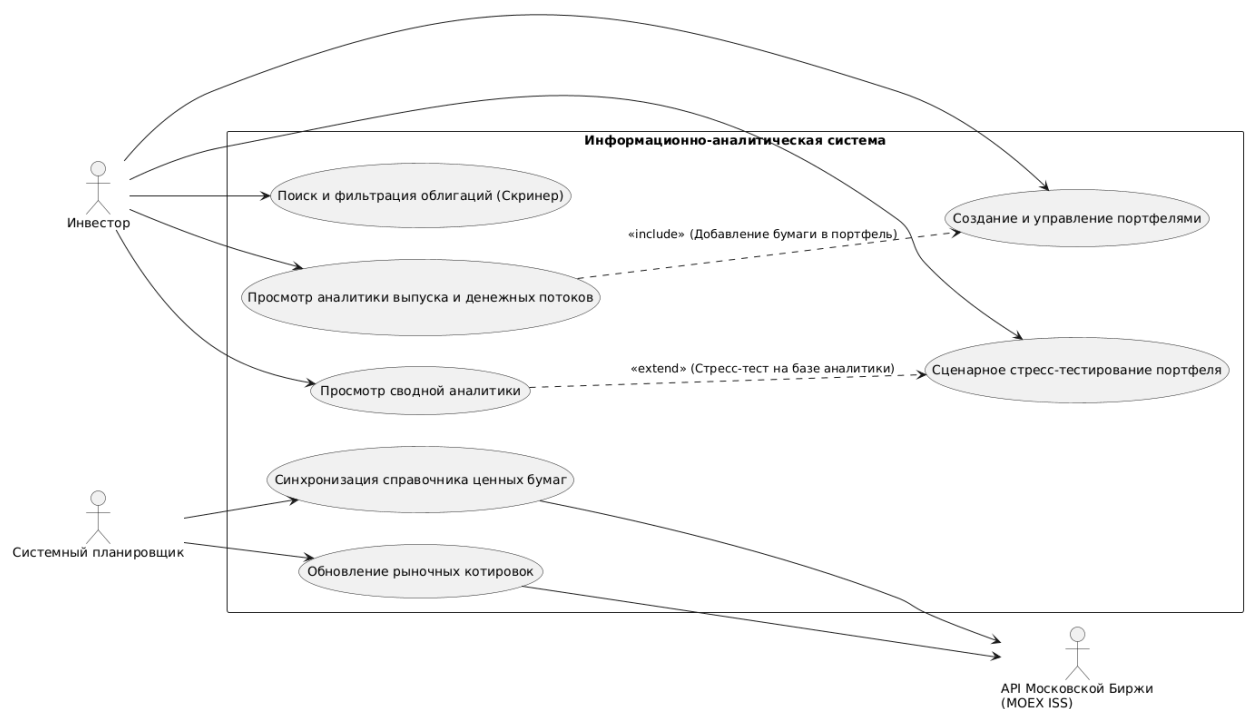


Рисунок 2.1.1 – Диаграмма вариантов использования проектируемой системы

На основе диаграммы из рисунка 2.1.1 формируются функциональные требования:

FR-1 Система должна позволять пользователю создавать портфели, добавлять и удалять позиции в портфелях и рассчитывать средневзвешенную цену покупки для вычисления показателя прибыли.

FR-2 Система должна предоставлять интерфейс поиска облигаций с фильтрацией по типу купона, сроку погашения и возможностью сортировки по названию, типу купона, цене и дате погашения

FR-3 Система должна рассчитывать и отображать показатели долговых бумаг. Расчет должен производиться на основе актуальных котировок.

FR-4 Система должна производить оценку стоимости портфеля при заданном пользователем изменении ключевой ставки.

FR-5 Система должна автоматически, с заданной периодичностью, извлекать актуальные котировки и данные с Московской биржи через API MOEX ISS, обрабатывать полученные их и вносить в базу данных.

К нефункциональным требованиям системы относятся:

NFR-1 Расчет метрик и стресс-теста из 50 уникальных бумаг должен занимать не более 500 мс для обеспечения плавной работы с минимальными задержками.

NFR-2 Архитектура приложения должна быть многоуровневой, со строгим разделением слоев: работа с данными, бизнес-логика, программный интерфейс, обеспечивая стабильность и масштабируемость системы, упрощение поддержки кода.

NFR-3 Структура взаимодействия компонентов должна соответствовать принципам модульного монолита. Логические блоки кода должны быть изолированы друг от друга, но физически запускаться в едином процессе. Такой подход позволит ускорить разработку, снизив сложность обработки зависимостей и архитектуры, и облегчит переход в будущем на микросервисную архитектуру [14].

NFR-4 Должна быть реализована идентификация пользователей для хранения их личных портфелей, исключая возможность несанкционированного доступа к портфелям других пользователей.

Для реализации заявленных требований был выбран следующий технологический стек.

На слой хранения данных была выбрана база данных PostgreSQL, выбор обоснован необходимостью работы с реляционными моделями.

Бизнес-логика реализована на языке программирования Python 3.14. Для математических вычислений использованы профессиональные библиотеки `numpy`, `scipy`, `pandas`. По нефункциональному требованию NFR-3 логика совмещена с программным интерфейсом API, поэтому был выбран Python фреймворк `FastAPI`, который обеспечивает асинхронность, авто генерацию OpenAPI-документации и поддерживает один из самых производительных серверных движков python – `uvicorn`.

Фоновый процесс актуализации котировок представлен инструментом `APScheduler`, он также работает в одном процессе с бизнес-логикой и программным интерфейсом, однако предполагается возможность выполнения через ETL системы или внешнюю систему очередей задач `Celery`, предназначенную для Python, но избыточны для минимально жизнеспособного продукта.

Для реализации пользовательского интерфейса были выбраны фреймворк `React` и язык программирования `TypeScript`. Эта комбинация является одним из лидеров в финансовой фронтенд разработке. `TypeScript` обеспечивает типизацию данных и их валидацию, а `React` имеет огромное количество нативных модулей, которые удобно использовать. В дополнение используются популярные библиотеки стилей и объектов `Tailwind CSS` и `Shadcn`.

2.2 Проектирование базы данных

При проектировании предметной области было принята концепция разделения структуры на базовую сущность ценных бумаг – securities и его объектов-сущностей облигации как класса ценных бумаг – bonds. Такой подход обеспечивает масштабируемость моделей, что позволит при необходимости добавлять другой класс – акции и потребуются лишь создать смежные таблицы stocks с реляционной связью с таблицей ценных бумаг, не нарушая логику связей портфелей и рыночных котировок облигаций [15].

База данных спроектирована с учетом 3-й нормальной формы для исключения дублирования данных. Основные таблицы и их назначения представлены в таблице 2.2.

Таблица 2.2 – Назначение таблиц баз данных

Название таблицы	Характеристики
	Назначение таблицы
securities	Базовая сущность. Первичным ключом выступает ISIN, что гарантирует уникальность записей
Bonds	Содержит специфичные для долговых бумаг атрибуты: номинал, дату погашения, валюту и тип купона
cash_flows	Хранит будущие выплаты с атрибутом типа потока. Позволяет корректно дисконтировать облигации с частичным погашением номинала
market_data	Хранит актуальные данные о торгах
portfolio	Портфели пользователей
portfolio_items	Позиции портфелей

Представление связей таблиц показано в ER диаграмме на рисунке 2.2.1.

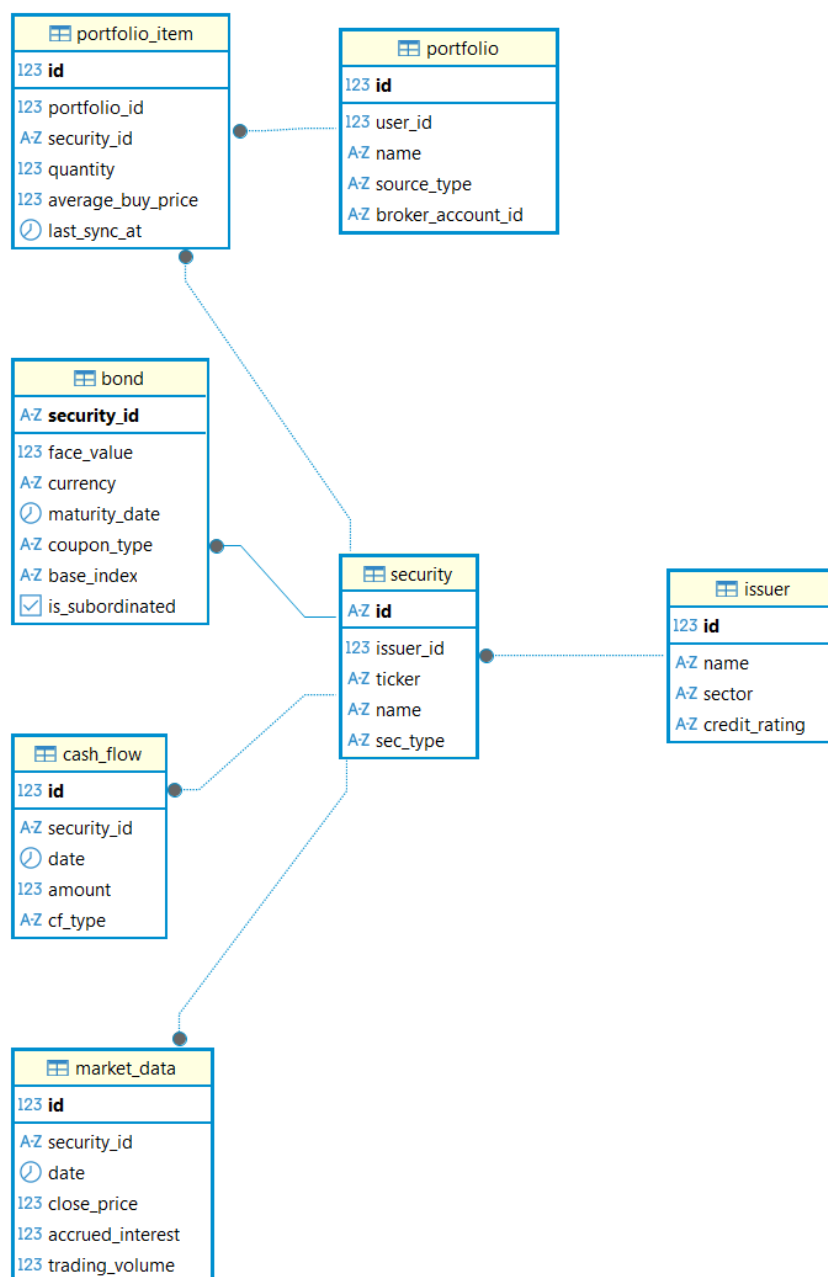


Рисунок 2.2.1 – ER модель схемы базы данных

2.3 Архитектура приложения и интеграция с внешними сервисами и системами

Ранее в технологическом стеке неявно были описаны архитектурные слои системы: слой представления (фронтенд/пользовательский интерфейс),

программный слой (бэкенд, бизнес-логика, API), сбор и обработка данных, хранение данных. В этой главе рассмотрим UML диаграммы, раскрывающие работу и взаимодействия слоев.

Программный слой спроектирован с применением чистой архитектуры и принципам SOLID. Основной задачей было снижение связности между компонентами системы для обеспечения её тестирования и масштабируемости.

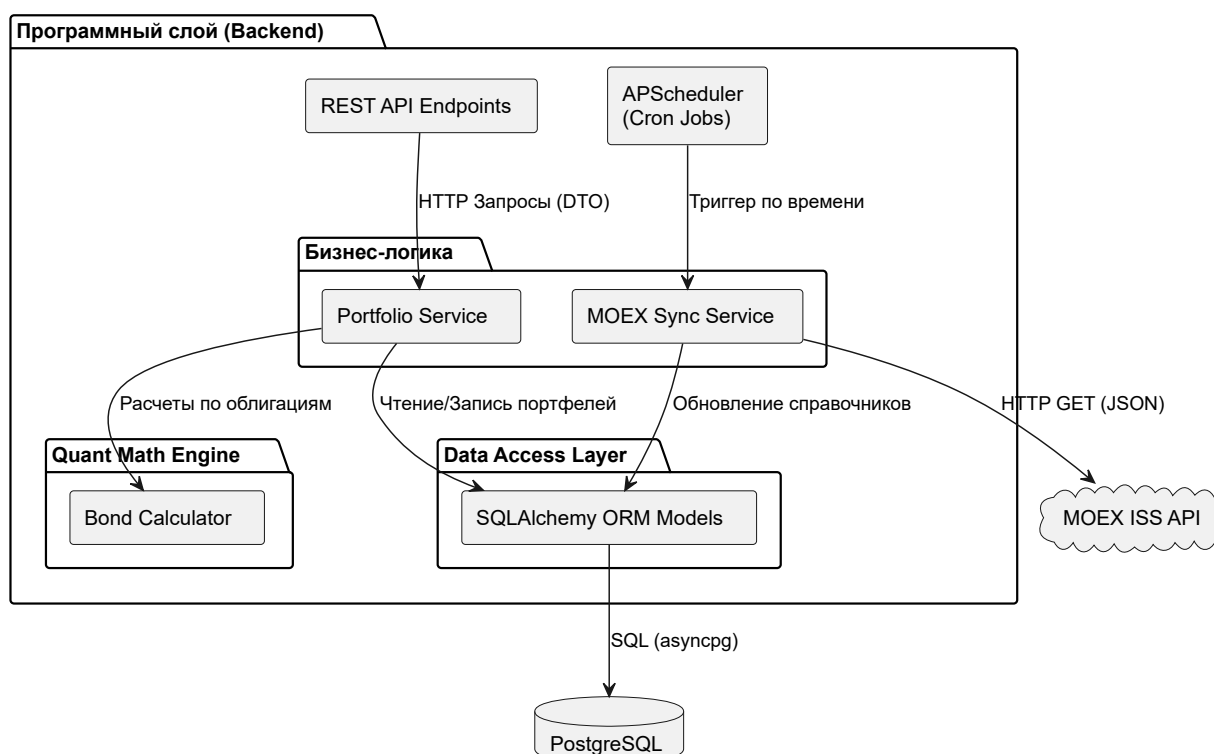


Рисунок 2.3.1 – Диаграмма компонентов программного слоя

Программный интерфейс Московской биржи MOEX ISS API позволяет любому пользователю, системе получать данные. Однако есть определенные ограничения: для получения данных в режиме реального времени необходимо приобрести подписку (без подписки данные обновляются с периодичностью 15 минут), полученные данные приходят в сыром текстовом виде. Первое ограничение критично для торговых и брокерских систем. В случае информационно-аналитической системы такое ограничение не является

критичным. Поэтому главной задачей остается обработка полученных данных для конвертирования в классы и модели. Все методы программного интерфейса предоставляются открыто с описанием, входными атрибутами и выходными атрибутами. Алгоритм обработка таких данных представлен на рисунке 2.3.2 [15].

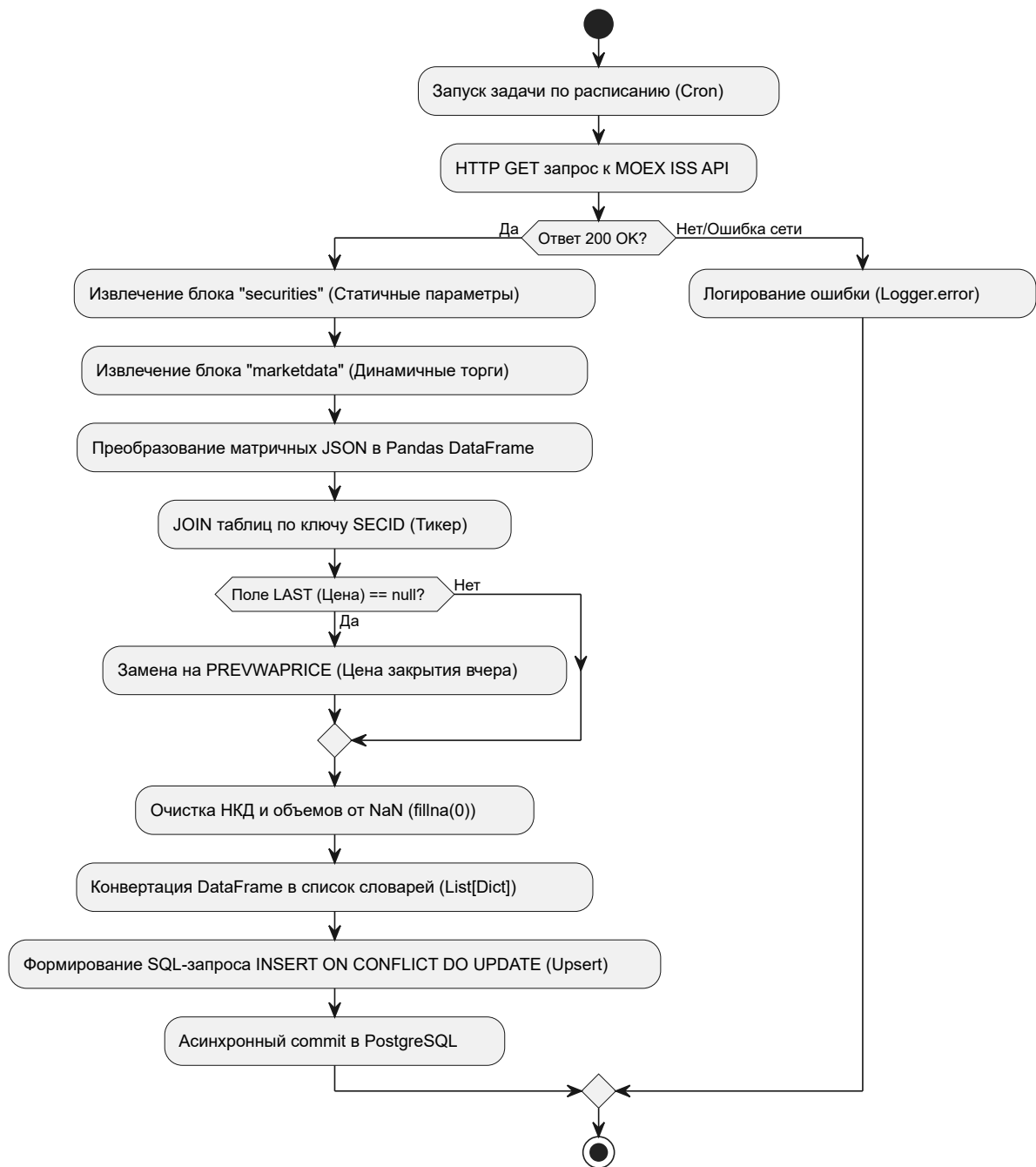


Рисунок 2.3.2 – Диаграмма активностей процесса синхронизации котировок

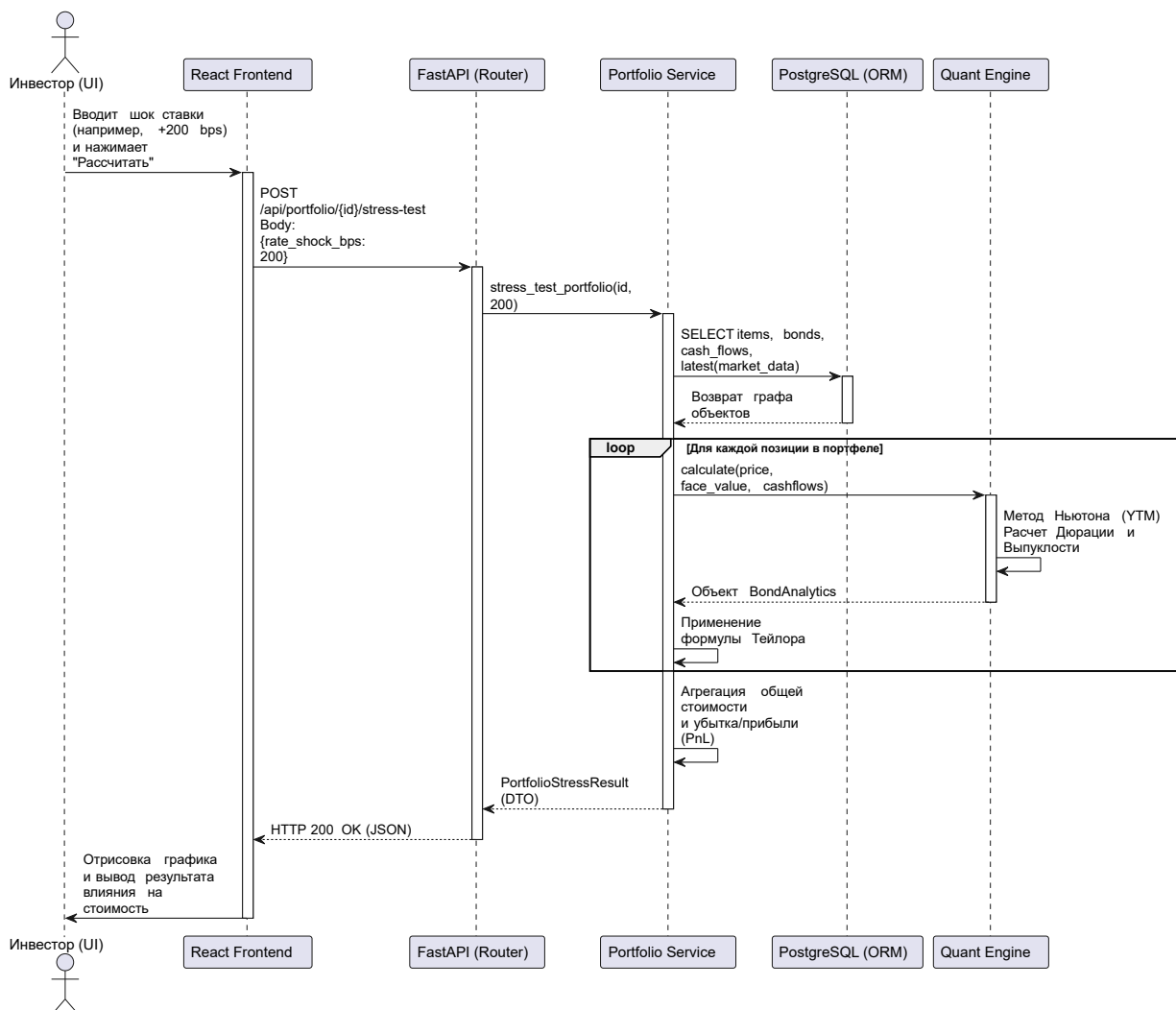


Рисунок 2.3.3 – Диаграмма последовательности стресс-тестирования портфеля

Одним из основных процессов в системе является модуль стресс-тестирования портфеля при макроэкономических шоках. Для этого при разработке была спроектирована последовательность операций модулей приложения, изображенная на рисунке 2.3.3.

Также для понимания взаимодействия пользователя с системой была спроектирована кросс-функциональная схема активностей, приближенная к описанию нотации бизнес-процессов. В схеме на рисунке 2.3.4 представлена последовательность действий пользователя и ответ на эти действия системы.

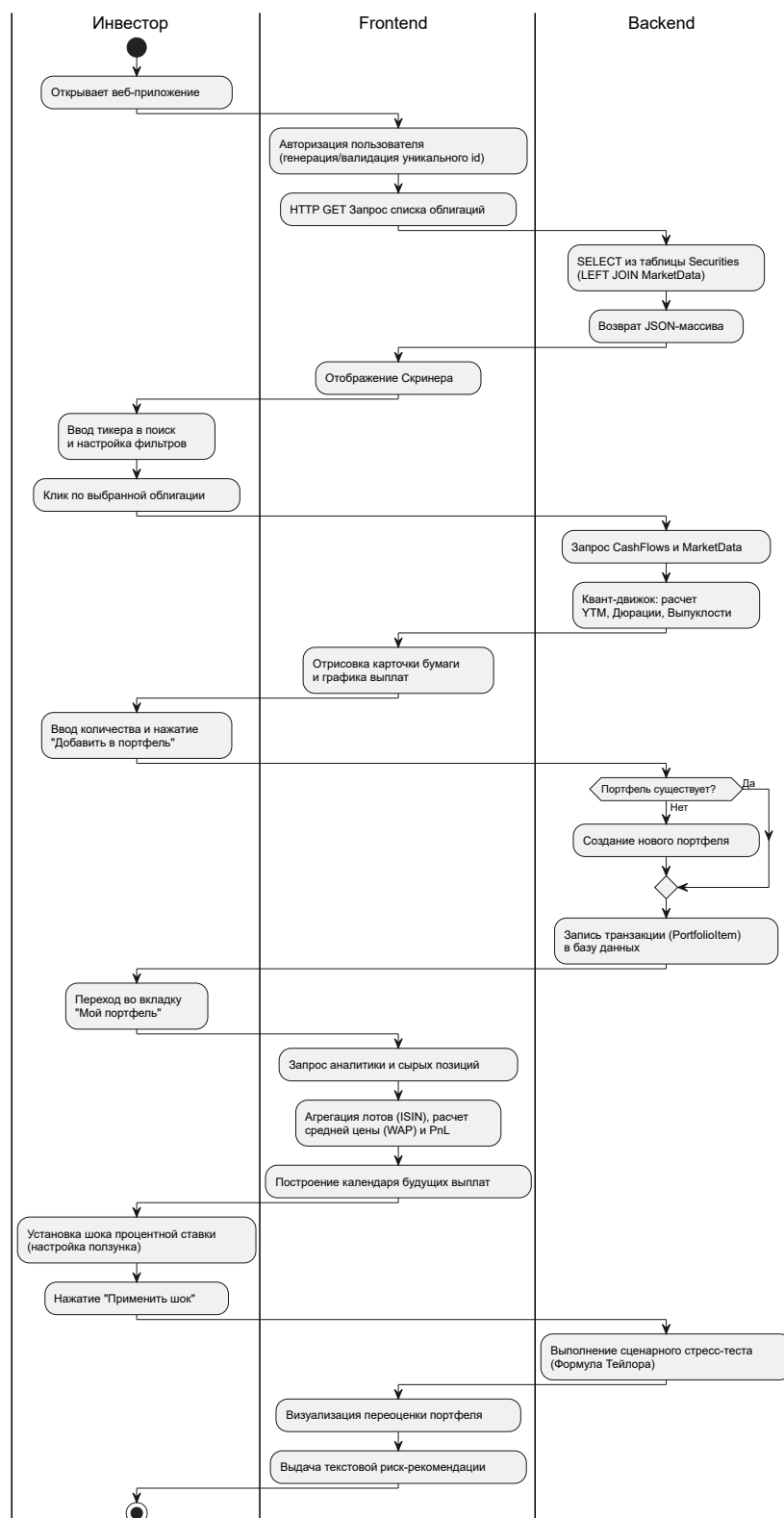


Рисунок 2.3.4 – Диаграмма бизнес-процесса взаимодействия с системой

3 Программная реализация

Верхнеуровнево по архитектуре приложение состоит из 3 основных компонентов: серверная часть/backend – центр логики системы; пользовательский интерфейс/frontend – компонент представления логических результатов системы в удобочитаемом для человека виде; база данных – постоянное хранилище данных. Про базу данных было описано в предыдущих главах. В этой главе будут рассмотрены процессы разработки серверной части и пользовательского интерфейса.

3.1 Разработка логики

Процесс разработки бэкенда был разбит на несколько последовательных и итеративных этапов:

- реализация доступа к данным из БД и преобразование в ORM-модели
- изолированный математический блок (Quant)
- интеграционный слой взаимодействия с MOEX ISS API и автоматизация работы с данными: получение, обработка, добавление, обновление
- реализация авторизации и портфельного блока
- добавление алгоритмов стресс-тестирования
- проектирование программных интерфейсов и бизнес-логики

Такое разделение позволило избежать связности кода и обеспечило возможность независимого тестирования каждого компонента.

Перед разработкой всей системы, по принципу разделенной ответственности был реализован класс настроек Settings, отвечающий за конфигурирование системы: параметры подключения сервисов, константные переменные, необязательные переменные, имеющие значения по умолчанию.

Класс настроек основан на базовом классе настроек BaseSettings из библиотеки pydantic. Pydantic модели также используются для валидации

данных, благодаря чему при несовпадении входящих данных будет выдана ошибка и не будут использоваться в бизнес-логике. Pydantic класс `BaseSettings` автоматически считывает переменные из внешнего файла `.env` или через переменные среды, применяя к ним аннотации типов данных. Если при инициализации данные не соответствуют указанному типу данных – будет вызвано исключение. Параметры и примеры переменных указываются в документации проекта, а также могут быть в файле шаблоне `.env.example`.

Для доступа других классов к переменным или методам конфигурации реализована функция `get_settings`, которая возвращает объект класса `Settings`:

```
def get_settings() -> Settings:
    return Settings()
```

3.1.1 Доступ к данным БД

Взаимодействие приложения с базой данных выполнено с использованием ORM-библиотеки `SQLAlchemy 2` и асинхронного драйвера `asynctrpg`. Выбор драйвера `asynctrpg` вместо классического синхронного `psycopg2` обусловлен его производительностью. Драйвер `asynctrpg` использует неблокирующий ввод-вывод и внутренний пул соединений, что позволяет обрабатывать множество параллельных запросов без создания дополнительных потоков операционной системы [13]. В традиционном синхронном веб-приложении каждый запрос к базе данных блокирует поток, что может привести к ошибкам других компонентов системы, которые также делают запросы к базе в этот момент. Одно из решений – масштабирование количества потоков, что приводит к увеличению затрат вычислительных ресурсов. Но такой подход невыгоден и неэффективен учитывая архитектуру фреймворка `FastAPI` [16].

Конфигурация подключения к базе вынесена в класс `Settings`. В конфигурации формируется строка подключения через динамический метод свойства класса:

```
class Settings(BaseSettings):
    ...
    @property
    def DATABASE_URL(self) -> str:
        return (
            f"postgresql+asyncpg://{self.POSTGRES_USER}:
{self.POSTGRES_PASSWORD}"
            f"@{self.POSTGRES_HOST}:
{self.POSTGRES_PORT}/{self.POSTGRES_DB}"
        )
```

На основе этого свойства создаются движок и сессии:

```
settings = get_settings()
engine = create_async_engine(settings.DATABASE_URL,
echo=False)
async_session = async_sessionmaker(engine,
class_=AsyncSession, expire_on_commit=False)
```

Параметр `expire_on_commit` в состоянии `False` отключает автоматическую инвалидацию загруженных объектов после коммита транзакции, уменьшает риск гонки данных и увеличивает производительность. Асинхронная сессия позволяет в момент ожидания ответа от базы данных переключиться на обработку других запросов. Для имплементации этой сессии в другие классы и объекты реализован генератор с автоматическим закрытием после завершения обработки:

```
async def get_db() -> AsyncSession:
    async with async_session() as session:
```

yield session

Все таблицы базы данных в коде представлены в виде классов, наследующих декларативный базовый класс Base из библиотеки SQLAlchemy. Данная библиотека является стандартом Object-Relation Mapping и позволяет связать таблицы с Python классами и работать с ними как с объектами. Рассмотрим представление таблицы cash_flow как класса в коде с использованием классов и методов библиотеки:

```
class CashFlow(Base):
    __tablename__ = "cash_flow"

    id: Mapped[int] = mapped_column(Integer,
primary_key=True, autoincrement=True)
    security_id: Mapped[str] = mapped_column(String(12),
ForeignKey("security.id"), nullable=False)
    date: Mapped[date] = mapped_column(Date,
nullable=False)
    amount: Mapped[float] = mapped_column(Float,
nullable=False)
    cf_type: Mapped[CashFlowType] =
mapped_column(Enum(CashFlowType), nullable=False)

    security: Mapped["Security"] =
relationship(back_populates="cash_flows")

    def __repr__(self) -> str:
        return f"<CashFlow(id={self.id},
date='{self.date}', cf_type='{self.cf_type}')>"
```

Конструкция Mapped[int] аннотацией типа и декларацией колонки для SQLAlchemy. Функция mapped_column принимает тип и параметры SQL

колонки таблицы: может ли быть пустой, тип ограничения, автоинкрементация. Для поля `cf_type` используется маппинг нестандартного типа `CashFlowType`, который задан произвольно представляет собой перечисление типов денежных потоков:

```
class CashFlowType(str, enum.Enum):  
    coupon = "coupon"  
    amortization = "amortization"  
    offer = "offer"
```

На уровне базы данных значения полей будут строковыми, но в коде используется такой класс для обеспечения надежности использования типов, исключая возможность опечатки при указании строк.

Из рисунка 2.2.1 таблица `cash_flow` имеет связь с таблицей `securities` по полю `security_id`. В коде такая связь реализована через метод `relationship` с параметром `back_populates`. Это также позволяет при загрузке ценной бумаги одним запросом подгрузить все связанные денежные потоки. Для модели `Security` настроено каскадное удаление – при удалении ценной бумаги все её денежные потоки и другие связанные с бумагой данные удаляются автоматически. Таким образом реализованы и другие таблицы и связи в форме ORM-моделей из представленной в прошлой главе схемы.

3.1.2 Математический блок

Математическое ядро спроектировано изолированно и без сохранения состояния. В такой реализации вычисления не зависят от внешних классов, переменных и подключений к базе данным, а входными данными являются родные структуры – числа, списки и время.

Класс `BondCalculator` осуществляет методы различных расчетов. Для удобства обработки входных данных были добавлены `pydantic` модели `CashFlowRecord` и `BondAnalytics`.

Первая модель необходима для передачи данных о купонных выплатах и содержит поля: `date` типа данных `datetime.date` – дата выплаты и `amount` типа данных `float` – количественное значение выплаты.

Вторая модель содержит в себе необязательные поля типа данных `float`: `ym` – доходность к погашению в процентах годовых, `macaulay_duration` – дюрация Маколея в годах, `modified_duration` – модифицированная дюрация, `convexity` – выпуклость, `dv01` – изменение цены при сдвиге ставки на один базисный пункт. И один обязательный элемент типа данных `float` `dirty_price` – грязная цена.

Первым этапом является преобразование денежных потоков в массив `numpy`. Методом `_filter_and_arrays` выполняется фильтрация потоков, дата которых строго позже даты оценки (текущей на момент расчета), формирует и возвращает два `numpy` массива со значениями времени до каждого платежа и размер этих выплат. Временной интервал вычисляется по стандарту АСТ/365.

```
@staticmethod
def _filter_and_arrays(
    cashflows: list[CashFlowRecord],
    current_date: datetime.date,
) -> tuple[np.ndarray, np.ndarray]:
    future = [cf for cf in cashflows if cf.date >
current_date] # будущие выплаты
    if not future:
        return np.array([]), np.array([])
    dates = np.array([cf.date for cf in future],
dtype="datetime64[D]") # массив дат выплат
    ref = np.datetime64(current_date)
    days = (dates - ref).astype(np.float64) # массив
дней до выплат с даты оценки
    t = days / 365.0
```

```

        cf_arr = np.array([cf.amount for cf in future],
dtype=np.float64)
        return t, cf_arr

```

Зная денежные потоки, можно реализовывать функции для вычисления доходности к погашению. Был выбран метод Ньютона-Рафсона и реализованы методы для функций NPV и её первой производной:

```

@staticmethod
def _npv(y: float, t: np.ndarray, cf: np.ndarray, price:
float) -> float:
    return float(np.sum(cf / (1.0 + y) ** t) - price)
@staticmethod
def _npv_prime(y: float, t: np.ndarray, cf: np.ndarray)
-> float:
    return float(np.sum(-t * cf / (1.0 + y) ** (t +
1.0)))

```

Сам метод есть в библиотеке `scipy` на вход которого указываются функция `_npv`, её производная `_npv_prime`, начальное приближение 10% с погрешностью до 8 знака после запятой и ограничение в 100 итераций, чтобы не было бесконечного цикла [17]. При выполнении функции применяются исключения возможных ошибок: обработка несходимости, некорректность заданных аргументов и переполнение потока. Если метод не сошелся, то возвращается значение `None`, в таком случае предполагается продолжение работы приложения, но клиенту не будут рассчитаны метрика YTM и показатели, связанные с доходностью.

Следующими реализованными методами являются метрики процентного риска: дюрация Маколея, модифицированная дюрация и выпуклость. Функции написаны по формулам (1.4.1), (1.4.2), (1.4.4) с

использованием массивов `numpy`, что позволило избежать использования циклов.

Общая функция расчета и публичный метод класса принимает на вход аргументы: чистая цена, номинал, НКД, объект-список денежных потоков и текущую дату оценки. Список аргументов соответствует тем данным, которые возвращает биржа. Последовательно вычисляются ранее описанные функции и результаты функций указываются в значения соответствующих элементов класса. Результат метода возвращает объект с заполненными полями.

3.1.3 Интеграция с Московской Биржей и обработка данных

Важной стадией разработки является процесс сбора данных с Московской биржи, их актуализация. Данную стадию можно разделить на три этапа: сбор данных по `http` через `MOEX ISS API`, обработка полученных данных и загрузка в БД [18].

На этапе извлечения данных был реализован базовый класс `BaseMoexClient`, который выполняет роль взаимодействия с программным интерфейсом биржи на транспортном уровне. В его основе заложены методы создания асинхронного клиента `httpx.AsyncClient`, формирования запроса к конечному узлу с указанием нужных заголовков, типа запроса, параметров и данных, инициализации и закрытия соединения. Применены методики повторных попыток запросов при возможных сетевых сбоях их логирование, как показано на рисунке 3.1.3.

```
2026-06-05 13:23:07 -[WARNING] - app.services.moex.base_client - MOEX request failed (attempt 1/3), retrying in 1s: /iss/engines/stock/markets/bonds/securities.json
2026-06-05 13:23:18 -[WARNING] - app.services.moex.base_client - MOEX request failed (attempt 2/3), retrying in 2s: /iss/engines/stock/markets/bonds/securities.json
2026-06-05 13:23:30 -[WARNING] - app.services.moex.base_client - MOEX request failed (attempt 3/3), retrying in 4s: /iss/engines/stock/markets/bonds/securities.json
2026-06-05 13:23:44 -[ERROR] - app.services.moex.base_client - MOEX request exhausted all 3 retries for: /iss/engines/stock/markets/bonds/securities.json
2026-06-05 13:23:44 -[ERROR] - app.services.moex.sync_service - Market data sync failed
```

Рисунок 3.1.3 – Вывод повторных попыток и ошибки соединения

Важным элементом является парсинг полученных данных, которые возвращаются московской биржей в формате блоков словарей (также можно назвать таблицами): у каждой таблицы есть колонки «metadata» - словарь, содержащий имена колонок и метайнформацию, «columns» - список названий столбцов, «data» - матрица значений по столбцам. Такие данные удобно преобразовать в датафрейм с помощью библиотеки pandas [18].

```
def _parse_to_dataframe(json_data: dict[str, Any],
block_name: str) -> pd.DataFrame:
    block = json_data.get(block_name)
    if not block or "columns" not in block or "data" not
in block:
        return pd.DataFrame()
    return pd.DataFrame(columns=block["columns"],
data=block["data"])
```

Этап обработки данных осуществляется в классе МоехAPI имплементирующий базовый класс клиента. Классу присваиваются константные значения эндпоинтов полям, которые определяют путь запроса для методов класса. Основные используемые пути:

а) /iss/engines/stock/markets/bonds/securities.json – возвращает таблицу инструментов торговой сессии по рынку облигаций.

б) /iss/securities/{isin}.json – возвращает спецификацию инструмента.

в) /iss/statistics/engines/stock/markets/bonds/bondization/{isin}.json – возвращает амортизации и купонные выплаты по инструменту.

г) /iss/history/engines/stock/markets/bonds/boards/{boardid}/securities/{secid}.json – история торгов инструмента.

д) /iss/history/engines/stock/markets/bonds/boards/{boardid}/yields/{secid}.json – история доходности инструмента.

Добавляются методы get_bond_info, get_bond_cashflow и get_market_data. Логика выполнения методов практически идентична: получает данные из конечного узла, обрабатывает пустых или некорректных

значений, преобразование типов данных из полученных строк, возвращает словарь объектов. Рассмотрим одну из обработок для метода `get_market_data` [19]:

```
sec_df = self._parse_to_dataframe(bonds_json,
"securities")
md_df = self._parse_to_dataframe(bonds_json,
"marketdata")
...
df = pd.merge(sec_df, md_df, on="SECID", how="inner") #
слияние информации и данных торгов в одну таблицу по тикеру
price_series = df.get("LAST", pd.Series(index=df.index,
dtype=float)) # проверка на существование столбца price.
# если его нет добавить столбцы средневзвешенной цены и
цена закрытия предыдущего дня
if "PREVWAPPRICE" in df.columns:
    price_series =
price_series.fillna(df["PREVWAPPRICE"])
if "PREVPRICE" in df.columns:
    price_series = price_series.fillna(df["PREVPRICE"])
```

При получении и данных по облигации возможен случай, когда колонка `LAST` возвращается пустой, если по бумаге на момент запроса не было торгов. В этом случае берется средневзвешенная цена или последняя цена закрытия предыдущего дня торгов. Также в этих данных хранятся данные НКД и объема торгов, при их отсутствии значение заменяется нулем.

3.1.4 Портфели и стресс-тестирование портфеля

Для реализации портфеля и его бизнес-логики был добавлен класс `PortfolioService`. Он представляет собой набор методов и не хранит в себе

данных, кроме сессии базы данных. Методы не выполняют вычислений, а используют подключенные внешние модули: математические операции через BondCalculator и подключение к БД через сессии.

Для оптимизации запросов по данным портфеля: позиции бумаг и их параметры, денежные потоки, текущие котировки и исторические данные, использовались приемы жадной загрузки:

```
stmt = (
    select(PortfolioItem)
    .where(PortfolioItem.portfolio_id == portfolio_id)
    .options(

selectinload(PortfolioItem.security).selectinload(Security.bond)

    )
)
```

Метод `selectinload` позволяет выгрузить данные из таблицы `security` связанные с запрашиваемым портфелем и облигации из таблицы `bond` связанные со всеми ценными бумагами портфеля. Этот механизм экономит время выполнения и разработки, не используя сырые SQL выражения.

Для стресс-тестирования портфеля используется метод класса `PortfolioService` `stess_test_portfolio`. Он принимает аргументы идентификатора портфеля и величину шока в базисных пунктах. Его аналитический алгоритм описан в главе 1.4 и программная последовательность выполнения показана на рисунке 2.3.3. Результатом его выполнения является объект данных `PortfolioStressResult` произвольной `pydantic` модели с полями: идентификатора портфеля, величина шока, общей текущей стоимости портфеля, общей стоимости портфеля с влиянием процентного риска, разница значений в валюте и в процентах.

Помимо метода стресс-тестирования данный класс реализует публичный метод получения аналитических данных портфеля без шока (пример запроса с жадной загрузкой относится к этому методу). Метод `get_portfolio_risk` реализует историческую оценку рисков по годовой истории цен.

3.1.6 Программный интерфейс API

Финальными этапами разработки бэкенда приложения являются добавление конечных узлов для взаимодействия извне с реализованными методами и классами и автоматизация сбора данных с биржи.

Для удобства маршруты были разделены на две группы: облигаций и портфеля. В первой группе представлены три эндпоинта в таблице 3.1.6.1:

Таблица 3.1.6.1 – Маршруты облигаций

Путь эндпоинта	Характеристики		
	HTTP Method	Параметры	Описание
<code>/api/bonds</code>	GET	search	Возвращает данные облигаций для скринера
<code>/api/bonds/{isin}</code>	GET	isin	Возвращает данные облигации по идентификатору
<code>/api/bonds/{isin}/cashflows</code>	GET	isin	Возвращает данные денежных потоков по идентификатору бумаги

Во второй группе представлены три аналитических конечных узла и узлы с CRUD операциями по портфелям и инструментов портфеля в таблице 3.1.6.2:

Таблица 3.1.6.2 – Маршруты портфелей

Путь эндпоинта	Характеристики			
	HTTP Method	Параметры	Тело запроса	Описание
/api/portfolio	GET	x-user-id		Список портфелей
/api/portfolio	POST	x-user-id	{ "name": "string" }	Добавление портфеля
/api/ portfolio/ {portfolio_id}	DELETE	portfolio_id x-user-id		Удаление портфеля
/api/ portfolio/ {portfolio_id} /items	GET	portfolio_id x-user-id		Получение элементов портфеля
/api/ portfolio/ {portfolio_id} /items	POST	portfolio_id x-user-id	{ "isin": "string", "quantity": 0, "average_buy_price": 0 }	Добавление элемента в портфель
/api/ portfolio/ {portfolio_id} /items/	DELETE	portfolio_id item_id x-user-id		Удаление элемента из портфеля

{item_id}				
/api/ portfolio/ {portfolio_id} /analytics	GET	portfolio_id x-user-id		Получить аналитические данные по портфелю

Продолжение таблицы 3.1.6.2

Путь эндпоинта	Характеристики			
	HTTP Method	Параметр ы	Тело запроса	Описание
/api/ portfolio/ {portfolio_id} /stess-test	POST	portfolio_id x-user-id	{ "rate_shock_bps": 0 }	Получить рассчитанные данные при стресс- тестировании

Временная система аутентификации реализована через чтение HTTP-заголовка X-User-ID. Такой подход был выбран в связи с ограниченным временем разработки и ресурсами. Он не обеспечивает безопасность, так как возможно подобрать идентификатор, но позволяет показать принцип работы личных портфелей. Решает задачу изоляции данных разных сессий без внедрения архитектурно сложных алгоритмов авторизации.

Помимо сервисных узлов реализован технический - /health, возвращающий статус активности веб-сервера. Являющийся одним из стандартов для мониторинга и отслеживания состояния работы приложения.

3.1.7 Автоматизация синхронизации данных

Так как рыночные котировки изменяются постоянно в ходе торгов то, чтобы аналитика портфеля оставалась актуальной необходимо автоматически обновлять данные. Решением этой задачи является использование периодического выполнения операций сбора и обновления данных. С помощью инструмента APScheduler были реализованы задачи синхронизации данных облигаций и котировок [20]. Класс МоехSyncService реализует и предоставляет метод для выполнения конвейера задач: запрос данных с биржи, парсинг и очистка и обновление таблиц в базе данных. Данный метод используется в конструкции класса TaskManager и инициализируется в функции add_job объекта AsyncIOScheduler библиотеки APScheduler:

```
class TaskManager:
    def __init__(self) -> None:
        self._scheduler = AsyncIOScheduler()
    def start(self) -> None:
        self._scheduler.add_job(
            sync_market_data_job,
            "interval",
            minutes=15,
            id="sync_market_data",
            replace_existing=True,
        )
        self._scheduler.start()
    def shutdown(self) -> None:
        self._scheduler.shutdown(wait=False)
```

Интервал выполнения задачи задан в 15 минут из-за ограничений биржи, снижения сетевых нагрузок и минимизации шанса получить блокировку.

Запуск и остановка планировщика интегрированы в жизненный цикл FastAPI при старте приложения и его остановке соответственно.

Данный этап завершает стадию разработки основной логики приложения. Следующей стадией является разработка пользовательского интерфейса и его взаимодействия с программным интерфейсом приложения.

3.2 Реализация пользовательского интерфейса

Клиентская часть приложения представляет собой одностраничное веб-приложение. Стилизация интерфейса выполнена с помощью фреймворка Tailwind CSS v4 и библиотеки визуальных компонентов Shadcn UI. Эта связка позволила осуществить реализацию визуально приятного и адаптивного дизайна [21][22][23].

За маршрутизацию отвечает библиотека React Router и содержит пути представления элементов скринера облигаций с поиском и фильтрами - /bonds, страница подробного описания отдельной облигации с графиком денежных потоков и формой для добавления её в портфель - /bonds/:isin и страница управления портфелями с аналитикой и стресс-тестированием - /portfolio.

Перед проектированием каждого из элементов были реализованы важные части интеграции клиента с серверной частью и аутентификации. Для взаимодействия с программным интерфейсом разрабатываемой системы использовалась библиотека axios, предоставляющая HTTP клиент и готовые полезные механизмы десериализация полученных JSON и типизации ответов через дженерики. Аналогично pydantic моделям серверной части в клиентской реализованы интерфейсы, повторяющие их структуру и типы данных. Такой подход соблюдает однородность структур данных и работу на клиенте и

сервер. Аутентификация на клиентской стороне реализована без явного ввода данных для входа. В основе этого подхода используется локальное хранилище браузера LocalStorage. При открытии приложения проверяется ключ `device_id` и его значение, если не было найдено, то генерируется случайное числовое значение из 9 знаков и сохраняется с этим ключом. Это значение задается, при каждом HTTP запросе через ранее добавленных `axios` клиент, в заголовке `X-User-ID`, который далее обрабатывается на сервере. Однако минусами такой реализации являются привязанность к локальному хранилищу браузерного клиента, то есть для любого другого браузера или даже профиля будет сгенерирован или использоваться новый идентификатор, и небезопасность, возможность подобрать чужой идентификатор.

3.2.1 Скринер и страница облигации

Скринер ценных бумаг представляет собой таблицу с поиском, дополнительными фильтрами и сортировками, которые помогают пользователю найти нужные инструменты. Таблица имеет столбцы данных: идентификатор облигации, название, эмитента, дату погашения, тип купона и цену в процентах от номинала. Данные запрашиваются функцией через программный интерфейс с сервера. Обязательным стало добавление пагинации таблицы, чтобы уменьшить объем отображаемых данных и снизить когнитивную нагрузку с пользователя. Поиск реализован через запрос к серверному API с параметром `search`, который ищет совпадения входного параметра в строковых значениях идентификаторов облигаций или названиях.

Стандартными в большинстве систем и доступными в разрабатываемой системе фильтрами являются отбор по типу купона: фиксированный, флоатер, линтер и бескупонный, и по сроку погашения: до 1 года, 1-3 года, 3-5 лет и более 5 лет. Сортировка реализована одним из стандартных способов – нажатием на заголовок столбцов таблицы. Активными столбцами,

наделенными сортировкой, стали «Тикер», «Погашение», «Тип купона» и «Цена». При нажатии на заголовок по умолчанию настраивается сортировка по убыванию, повторное нажатие приведет к изменению порядка по возрастанию и третьим нажатием сортировка вернется в исходное состояние без сортировки.

Просмотр подробной информации о бумаге реализован через скрытый столбец в таблице для добавления активной кнопки «Подробнее». При клике на кнопку осуществляется переход по шаблону маршрута /bonds/:isin. Значение isin берется из той же таблицы. Данная страница представляет собой информативную и аналитически-графическую информацию по облигации, её показателям и денежным потокам. Страница разделена на блоки числовых и строковых показателей, график денежных потоков и форма добавления в портфель. Аналогично ИАС Smart-lab в первом блоке размещаются показатели одного экземпляра инструмента. Были добавлены текущая цена в валюте, НКД, доходность к погашению, модифицированная дюрация и выпуклость, изображенные на рисунке 3.2.1.1.

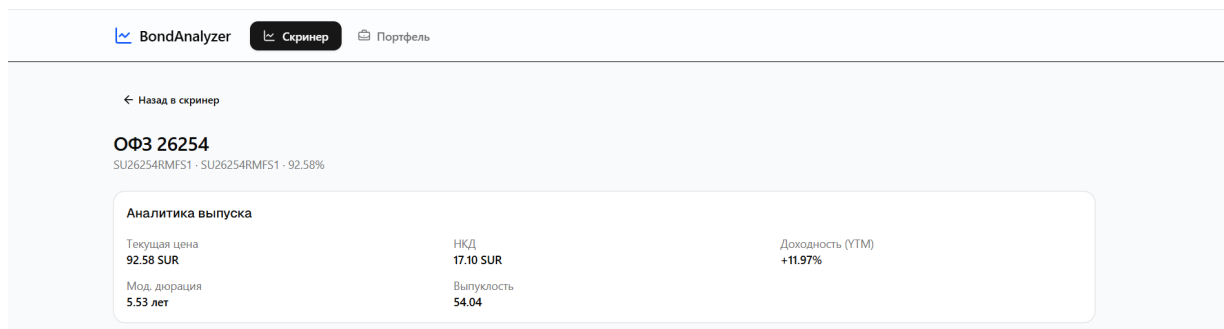


Рисунок 3.2.1.1 – Блок аналитических данных облигации SU26254RMFS1

В следующем блоке представлен колоночный график будущих денежных потоков одной бумаги, на рисунке 3.2.1.2. По оси X расположены даты выплат, по оси Y размеры выплат, каждая колонка соответствует одной выплате. При наведении на столбец отображается подсказка с точной датой и суммой выплаты.

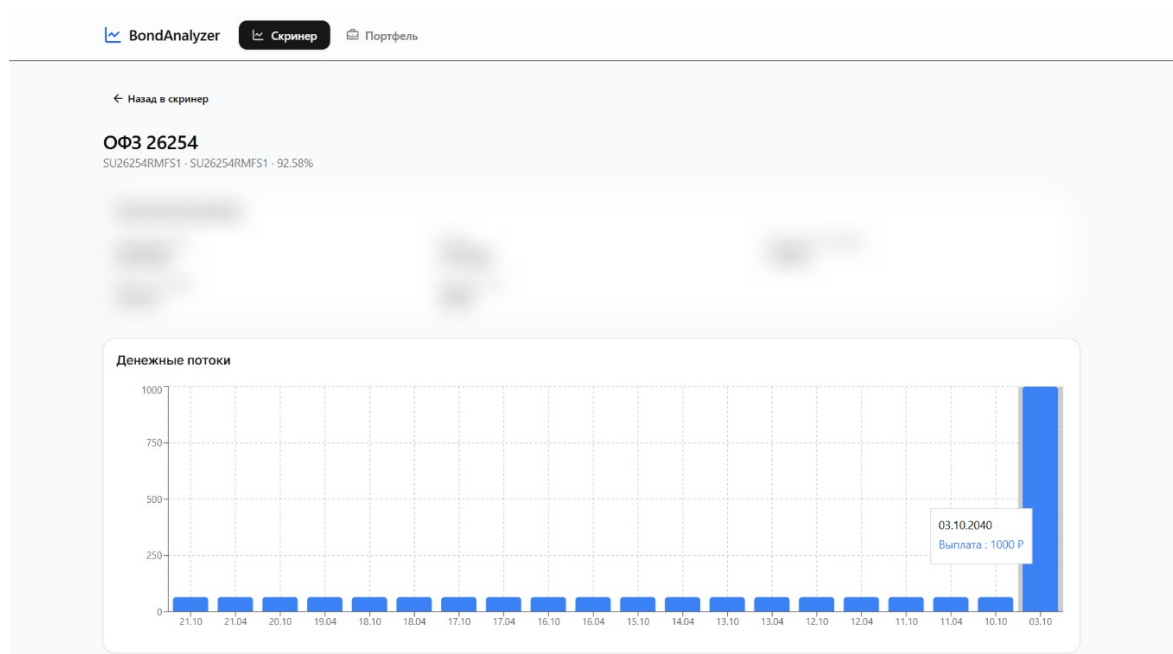


Рисунок 3.2.1.2 – График денежных потоков облигации SU26254RMFS1

Последний блок является формой добавления облигации в портфель – рисунок 3.2.1.3. Форма имеет 2 постоянных поля и одно зависимое от существования портфелей. Поле количества добавляемых в портфель инструментов принимает целочисленные значения не менее единицы. Цена покупки указывается в процентном соотношении от цены номинала, по умолчанию устанавливается текущая цена. Для удобства по полю «Цена покупки» выводится стоимость в валюте. Если пользователь имеет несколько профилей, то добавляется поле выбора портфеля с выпадающим меню. После нажатия на кнопку «Добавить в портфель» и успешного добавления инвестора переводит на страницу портфелей.

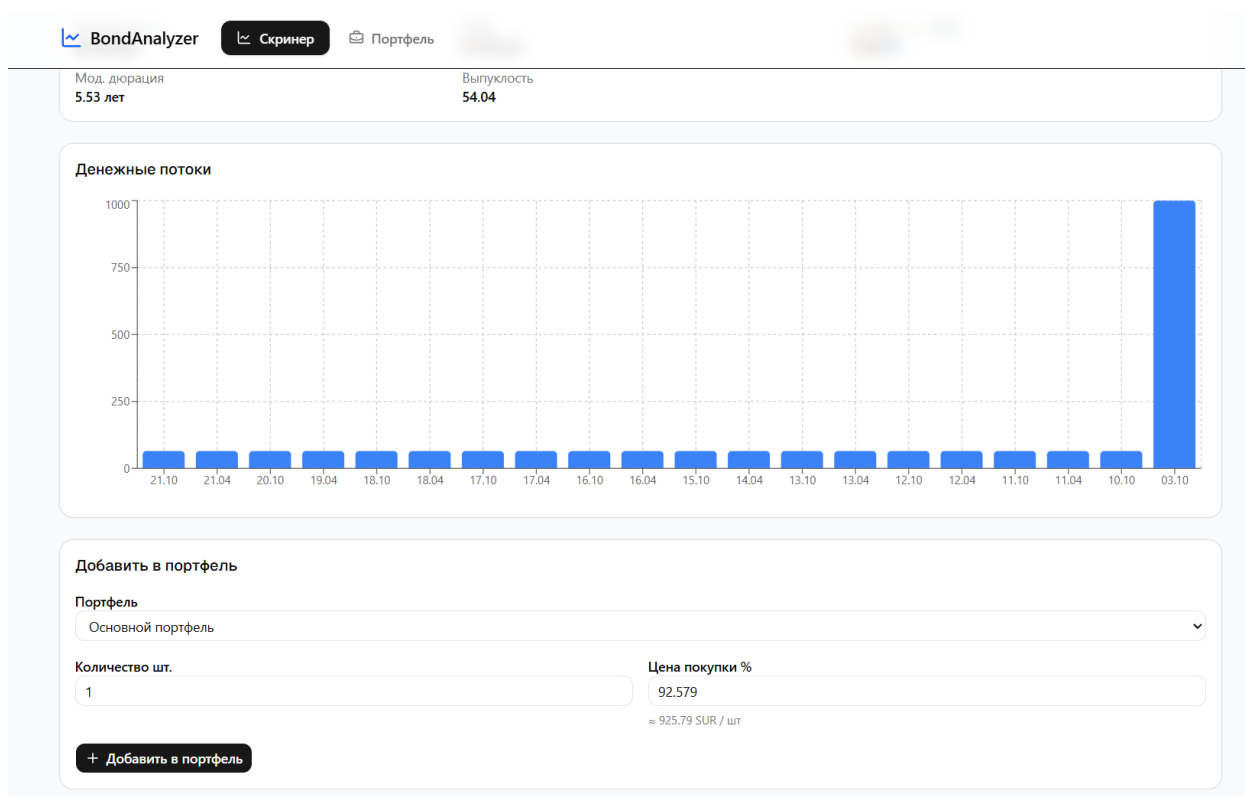


Рисунок 3.2.1.3 – Форма добавления позиции в портфель

3.2.2 Портфель

На странице портфеля главными компонентами являются агрегированные показатели для каждого инструмента и по всем позициям в портфеле. Общая информации о текущей стоимости всего портфеля, средней доходности и дюрации выведена в начале страницы. Также в таком окне помещается и добавлен общий колоночный график по всем будущим выплатам. Это информация необходима для представления пользователю совокупности определенных показателей по добавленным бумагам в портфель. На рисунке 3.2.2.1 изображена реализация страницы.

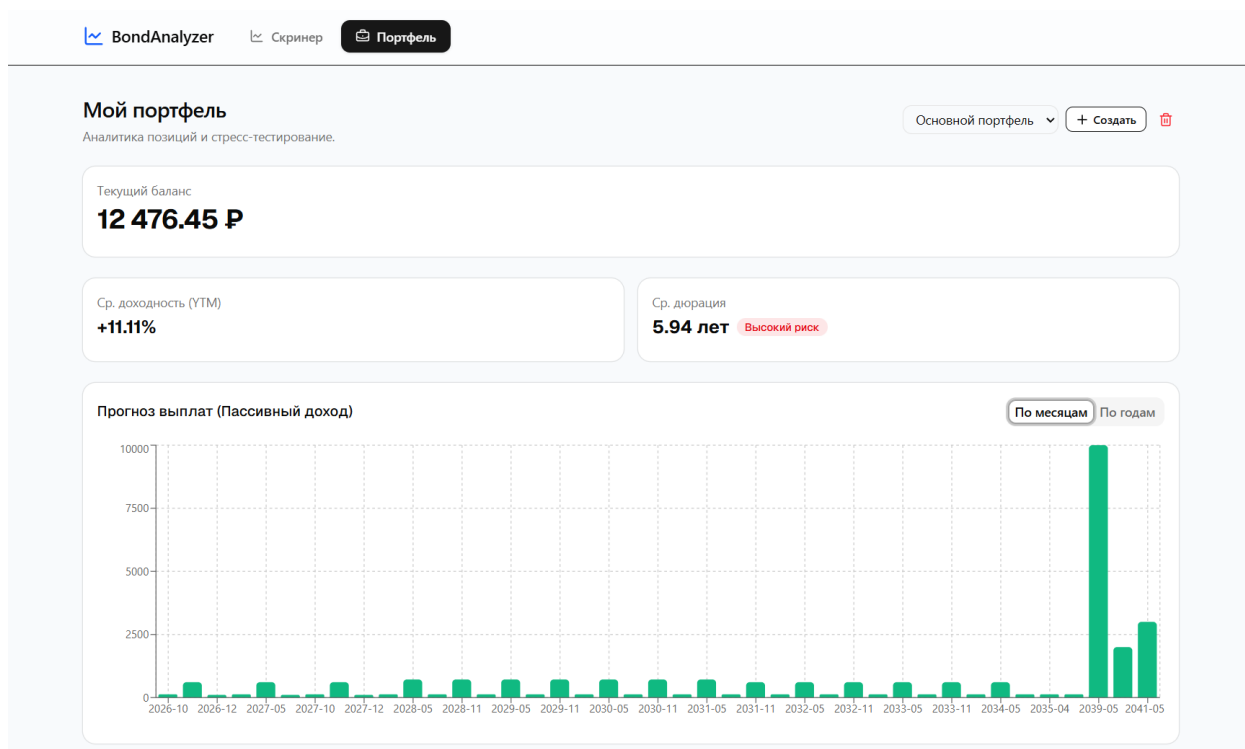


Рисунок 3.2.2.1 – Начальная страница портфеля

Важным элементом этой страницы является выбор, создание и удаление портфеля в правом верхнем углу. Переключение профилей осуществляется через выбор нужного элемента в селекторе. Создание вызывает диалоговое окно-форму, в котором можно указать название портфеля и нажать кнопку создать, как показано на рисунке 3.2.2.2. Для выхода из этого окна можно его закрыть, нажав на крестик или на пустую область возле самого окна. Для удаления также используется диалоговое окно, его можно вызвать, нажав на красную иконку корзины. Вызванное окно служит подтверждение удаления профиля и предотвращается выполнение по неосторожности или случайному нажатию.

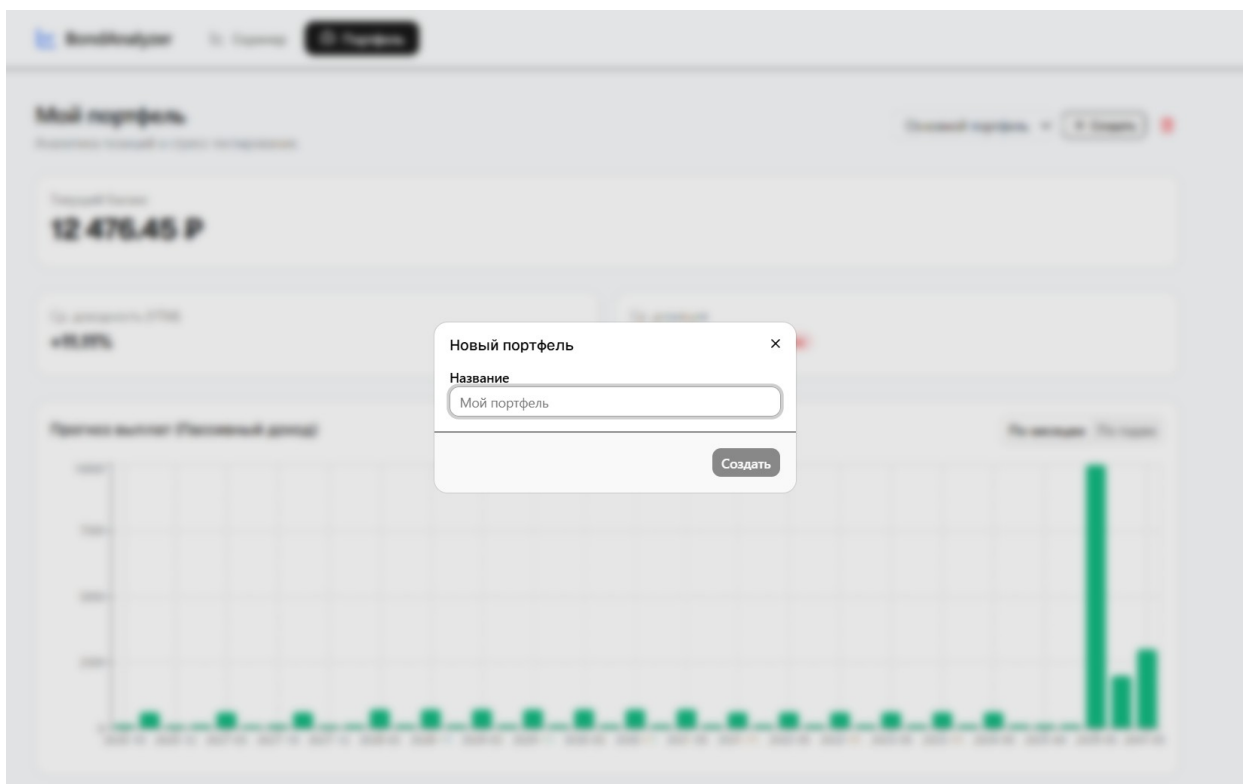


Рисунок 3.2.2.2 – Окно создания портфеля

Ниже начального экрана располагается блок аналитических показателей, агрегированных по каждой позиции сгруппированных по идентификатору инструмента. Реализация которого выполнена в таблице со столбцами: «Название» - идентификатор, «Кол-во» - количество добавленных бумаг, «Цена» - грязная цена одной штуки, «Ср. цена» - средневзвешенная цена покупки, «PnL» - бумажная (нереализованная) прибыль или убыток (показывает изменение стоимости позиции относительно средней цены покупки), «Стоимость» - общая цена всех бумаг, «УТМ» и «Дюрация». Также добавлена колонка удаления целой позиции из портфеля. Таблица показана на рисунке 3.2.2.3.

Позиции								
Название	Кол-во	Цена	Ср. цена	PnL ⓘ	Стоимость	YTM ⓘ	Дюрация ⓘ	Действия
ОФЗ 26238 SU26238RMFS4	3	572.71 Р	57.98%	-1.22%	1 718.13 Р	+8.67%	8.11 лет	
ОФЗ 26247 SU26247RMFS5	10	886.86 Р	88.10%	+0.67%	8 868.60 Р	+11.40%	5.60 лет	
ОФЗ 26254 SU26254RMFS1	2	942.60 Р	92.53%	+1.86%	1 885.20 Р	+11.97%	5.53 лет	

Рисунок 3.2.2.3 – Таблица аналитических показателей позиций портфеля

При оформлении в явном виде не указывались длинные названия и описания показателей, но были добавлены в сноски к заголовкам столбцов, изображённые на рисунке 3.2.2.4.

Позиции								
Название	Кол-во	Цена	Ср. цена	PnL ⓘ	Стоимость	YTM ⓘ	Дюрация ⓘ	Действия
ОФЗ 26238 SU26238RMFS4	3	572.71 Р	57.98%	-1.22%	1 718.13 Р	+8.67%	8.11 лет	
ОФЗ 26247 SU26247RMFS5	10	886.86 Р	88.10%	+0.67%	8 868.60 Р	+11.40%	5.60 лет	
ОФЗ 26254 SU26254RMFS1	2	942.60 Р	92.53%	+1.86%	1 885.20 Р	+11.97%	5.53 лет	

Доходность к погашению. Показывает реальную годовую доходность с учетом реинвестирования купонов.

Рисунок 3.2.2.4 – Сноска с описанием показателя таблицы

Последним компонентом пользовательского интерфейса является блок макроэкономического стресс-тестирования, находящийся в нижней части страницы. Его реализация представляет собой ползунок с граничными параметрами в -5.00% и +5.00% и шагом в 0.25% симуляции изменения ключевой ставки ЦБ РФ. Используются процентные значения для удобства конечного пользователя, в запросах и вычислениях используются значения базисных пунктов. Для применения выставленного ползунка используется кнопка «Применить шок», которая вызывает функцию отправки запроса с параметром показателя шока на серверную вычислительную логику и получает ответ с результатом, который выводит под формой. При отправке запроса и ожидания ответа кнопка становится неактивной, предотвращая повторные попытки до получения ответа от сервера. Под формой выводятся

данные новой баланса – прогнозируемая стоимость портфеля в валюте, дельта изменения с текущим балансом в валюте и процентном соотношении. Визуально при положительном изменении значения выделяются зеленым цветом и иконкой графика роста, в противоположном случае используется красный цвет и график падения.

На этом этапе разработка пользовательского интерфейса закончена. Выполнены функциональные требования FR-1, FR-2, FR-3, FR-4 и частично нефункциональное требование NFR-4, так используется неполноценная авторизация, а концепт.

3.3 Тестирование системы

Важным этапом разработки любого продукта является тестирование. В рамках этого процесса были реализованы модульное тестирование математического ядра, тестирование сервиса стресс-тестирования и функциональное тестирование конечных узлов. Модульное тестирование покрывает четыре сценария.

Случай проверки корректности вычисления цены нормальной облигации. Данный тест рассматривает на входе корректно заданную облигацию номинала 1000 рублей с годовым купоном 10% и торгующуюся ценой 95% от номинала. К облигации создается денежный поток с шестью полугодовыми купонными выплатами с помощью `pydantic` класса. Цель проверки совпадение вычисления грязной цены математическим модулем с заданным верным. Также проверяет, что доходность к погашению превышает купонную ставку, что является фундаментальным свойством дисконтных облигаций.

Второй случай аналогичен первому, но в этом тесте у облигации задается цена торгов выше номинала 105%. В этом случае должны получиться обратные результаты доходности – должна быть ниже купонной ставки.

Одним из возможных граничных случаев на рынке, может быть, отсутствие денежных потоков. Соответственно в этом сценарии подается пустой массив выплат. Корректное выполнение теста заключается в расчёте цены бумаги, а другие показатели вернуть пустыми. Тест считается выполненным при успешном вычислении и возвращении ответа без ошибок.

Четвертым сценарием является аномальные входные данные. В данном случае на вход поступает облигация с отрицательным числовым значением цены от номинала, например -200%. В реальности на рынке встретить такие показатели невозможно, но при получении данных из внешнего источника такие ситуации могут произойти или при ручном добавлении используя кодовые структуры. В этом случае система должна вернуть пустой объект, не переходя к вычислению показателей инструмента.

Тест модуля стресс-тестирования заключается в проверке четырех пунктов при следующих входных данных: портфель с одной позицией – 10 облигаций с ценой равной номиналу, трёхлетний срок до погашения, полугодовые купоны доходностью 10% годовых и применяет шок на +200 базисных пунктов. Текущая стоимость должна портфеля должна соответствовать 10000 рублей. При положительном шоке стоимость портфеля должна уменьшиться и в этом случае относительная разница портфелей должна быть отрицательной. Величина изменения в процентах, вычисленная должна совпадать с указанным значением, рассчитанным ранее подстановкой в используемые формулы. Данный тест проверяет бизнес-логику выполнения процесса применения стресса и корректность вычисления по формулам.

Одно из нефункциональных требований NFR-3 заключалось в проверке производительности системы, поэтому для этого был написан автоматизированный нагрузочный тест. Выполнение теста соответствует условиям требования и результат, изображенный на рисунке 3.3.1, выполняет заданное требование и выполнен за 15-20 мс.



Рисунок 3.3.1 – Результат выполнения нагрузочного тестирования

Тестирование каждого конечного узла программного интерфейса покрываются двумя случаями: корректные входные данные – получить успешный ответ, некорректные входные данные – вернуть ответ и статус ошибки.

Тесты выполнялись с помощью фреймворка pytest, выполнение изображено на рисунке 3.3.2 [24].



Рисунок 3.3.2 – Выполнение тестов

ЗАКЛЮЧЕНИЕ

В заключении данной выпускной квалификационной работы выполнена поставленная цель – спроектировано и реализовано веб-приложение для моделирования цен, расчета доходности и стресс-тестирования портфелей облигаций. Особое внимание было уделено процессу разработки – соблюдение архитектурных принципов, обеспечивающих расширяемость и надежность системы, а также итеративного подхода разработки, который позволил создать минимально жизнеспособный продукт. Для решения поставленных задач было выполнено следующее:

- проведён системный анализ предметной области, обзор существующих решений для розничных инвесторов и формализация математического аппарата дисконтирования денежных потоков облигаций: изучена структура рынка облигаций Московской биржи, проанализированы функциональные возможности брокерских мобильных приложений и профессиональных терминалов, выявлен разрыв в доступности инструментов оценки риска, формализована модель приведённой стоимости денежных потоков как основа для всех последующих вычислений;

- выполнено проектирование реляционной базы данных и разработка общей архитектуры приложения, разделяющей серверное вычислительное ядро, источник рыночных данных и клиентский интерфейс, спроектированы API узлы между клиентом и сервером, выделен фоновый планировщик APScheduler для периодической синхронизации котировок;

- осуществлена программная реализация серверной части с подсистемами сбора рыночных котировок через программный интерфейс Московской биржи, расчёта доходности к погашению численным методом Ньютона, разработан сервис стресс-тестирования с мгновенным параллельным шоком кривой доходности и агрегацией результатов по портфелю;

– разработан пользовательский интерфейс, обеспечивающий визуализацию позиций на странице портфеля, календаря купонных выплат как отдельного долгового инструмента, так и будущих выплат всего портфеля, результатов стресс-теста;

– проведено автоматизированное модульное и нагрузочное тестирование вычислительного ядра, подтверждающее корректность аналитических расчётов - реализован набор тестов, покрывающий необходимые проверки стабильности и корректности вычислений;

Результаты работы обладают практической ценностью. Готовое приложение дает розничному инвестору инструмент, позволяющий без специальной подготовки оценить, насколько его облигационный портфель устойчив к изменению ключевой ставки. Процесс выполнения работы позволил на практике осуществить ряд технологических практик проектирования и разработки работоспособной системы.

—

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Fabozzi, F.J. Bond markets, analysis, and strategies / F. J. Fabozzi, F. A. Fabozzi. — 10th ed.. — CambridgeLondon : MIT Press, 2021. — 922 p. — ISBN 978-0-262-04627-5
2. Федеральный закон от 22.04.1996 N 39-ФЗ (ред. от 09.04.2026) "О рынке ценных бумаг" (с изм. и доп., вступ. в силу с 20.04.2026) – Текст : электронный // КонсультантПлюс : справочно-правовая система : сайт. – URL: https://www.consultant.ru/document/cons_doc_LAW_10148/ (дата обращения 09.02.2026)
3. Федеральный закон от 22.04.1996 N 39-ФЗ (ред. от 09.04.2026) "О рынке ценных бумаг" (с изм. и доп., вступ. в силу с 20.04.2026). Глава 5. ЭМИССИЯ ЦЕННЫХ БУМАГ – Текст : электронный // КонсультантПлюс : справочно-правовая система : сайт. – URL: https://www.consultant.ru/document/cons_doc_LAW_10148/a83e1160d46a567e1fdcb347500dbb945e0ad156/ (дата обращения 09.02.2026)
4. Центральный банк Российской Федерации : [официальный сайт]. – URL : <https://www.cbr.ru/> (дата обращения 11.12.2025)
5. Московская биржа : [официальный сайт]. – URL : <https://www.moex.com/> (дата обращения 25.07.2025)
6. Теплова, Т. В. Инвестиции : учебник и практикум для вузов / Т. В. Теплова. — 2-е изд., перераб. и доп. — Москва : Издательство Юрайт, 2026. — 781 с. — (Высшее образование). — ISBN 978-5-534-18289-7.
7. Рынок ценных бумаг : учебник для вузов / под общей редакцией Н. И. Берзона. — 6-е изд., перераб. и доп. — Москва : Издательство Юрайт, 2026. — 519 с. — (Высшее образование). — ISBN 978-5-534-20549-7.
8. Методика расчета НКД и доходности // Московская Биржа : [официальный сайт]. — 2025. — URL : <https://www.moex.com/files/43mmvs4t00f21twfsmh91x4hxy> (дата обращения 14.03.2026)

9. Frederick R. Macaulay, Some Theoretical Problems Suggested by the Movements of Interest Rates, Bond Yields and Stock Prices in the United States since 1856 / Frederick R. Macaulay. — NBER, 1938. — 600 p. — ISBN 0870140329

10. Халл, Д.К. Опционы, фьючерсы и другие производные финансовые инструменты : перевод с английского / Д. К. Халл. — 8-е изд.. — МоскваСанкт-Петербург : Диалектика, 2019. — 1070 с.: ил., табл.. — Перевод издания: Options, futures, and other derivatives. — ISBN 978-5-907144-35-4.

11. Клеппман, М. Высоконагруженные приложения : программирование, масштабирование, поддержка : перевод с английского / М. Клеппман. — Санкт-ПетербургМоскваЕкатеринбург : Питер, 2019. — 638 с.: ил.. — (Бестселлеры O'Reilly). — Библиогр. в конце главПеревод издания: Designing data-intensive applications: the big ideas behind reliable, scalable, and maintainable systems. — ISBN 978-5-4461-0512-0.

12. Шаблоны корпоративных приложений : перевод с английского / М. Фаулер, Д. Райс, М. Фоммел [и др.]. — Испр. изд.. — МоскваСанкт-Петербург : Диалектика, 2020. — 543 с.: ил.. — (The Addison Wesley Signature Series). — Библиогр.: с. 527-531Перевод издания: Patterns of enterprise application architecture. — ISBN 978-5-6041394-0-0

13. Коберн, А. Современные методы описания функциональных требований к системам : перевод с английского / А. Коберн. — Москва : Лори, 2022. — 263 с.: ил.. — Библиогр.: с. 262-263Перевод издания: Writing effective use cases. — ISBN 978-5-85582-326-4

14. Ньюмен, С. Создание микросервисов / С. Ньюмен; перевод с английского С. Черников. — 2-е изд.. — Санкт-ПетербургМоскваМинск : Питер, 2024. — 622 с.: ил.. — (Бестселлеры O'Reilly). — Библиогр.: с. 610-614Перевод издания: Building microservices: designing fine-grained systems. — ISBN 978-5-4461-1145-9

15. PostgreSQL 18 Documentation // PostgreSQL : [официальный сайт]. – URL : <https://www.postgresql.org/docs/current/index.html> (дата обращения 10.08.2025)
16. FastAPI // FastAPI : [официальный сайт]. – URL : <https://fastapi.tiangolo.com/> (дата обращения 04.01.2026)
17. SciPy User Guide // SciPy : [официальный сайт]. – URL : <https://docs.scipy.org/doc/scipy/tutorial/index.html#user-guide> (дата обращения 10.03.2026)
18. Описание методов | ISS MOEX API // Московская Биржа : [официальный сайт]. – URL : <https://iss.moex.com/iss/reference/> (дата обращения 25.06.2025)
19. Маккинни, У. Python и анализ данных : первичная обработка данных с применением pandas, NumPy и Jupiter / У. Маккинни; перевод с английского А. А. Слинкин; редактор Д. А. Мовчан. — 3-е изд.. — Москва : ДМК Пресс, 2023. — 536 с.: ил., рис., граф., табл.. — Предм. указ.: с. 518Перевод издания: Python for data analysis: data wrangling with pandas, NumPy & Jupyter. — ISBN 978-5-93700-174-0.
20. User guide – APScheduler // APScheduler : [официальный сайт]. – URL : <https://apscheduler.readthedocs.io/en/3.x/userguide.html> (дата обращения 12.02.2026)
21. React Reference // React : []. – URL : <https://react.dev/reference/react> (дата обращения 14.10.2025)
22. TypeScript: The starting point ... // TypeScript : [официальный сайт]. – URL : <https://www.typescriptlang.org/docs/> (дата обращения 14.10.2025)
23. Tailwind CSS with Vite // Tailwind CSS : [официальный сайт]. – URL : <https://tailwindcss.com/docs/installation/using-vite> (дата обращения 14.10.2025)
24. pytest documentation // Pytest : [официальный сайт]. – URL : <https://docs.pytest.org/en/stable/> (дата обращения 28.09.2025)